Payment Reconciliation & Audit Trail System

Complete Enterprise Learning Guide

From Absolute Beginner to Enterprise Architect

Domain Experts Contributing
FinTech Domain Expert | Chartered Accountant | Banking Operations Expert
Enterprise Solution Architect | Senior Backend Engineer | Senior Product Manager
Data Architect | Audit & Compliance Expert

PART 1 — THE BIG PICTURE: What Is Payment Reconciliation?

1.1 What Is Payment Reconciliation?

Payment Reconciliation is the process of comparing two independent sets of financial records to verify they are consistent, correct, and complete. Think of it as a double-check: you compare what your internal system says happened with what an external source (like a bank or payment gateway) confirms actually happened.

> **Simple Analogy**
> Imagine you go to a shop and buy items worth ₹500. The shopkeeper's till shows ₹500. Your bank statement also shows ₹500 debited. When both match — that is reconciliation. In enterprises, millions of such transactions happen daily and every single one must match.

The Core Idea in One Sentence

Reconciliation answers the question: 'Does our internal book of records exactly match the external source of truth?'

1.2 Why Do Companies Need It?

Every business that handles money faces the same fundamental problem: data is recorded in multiple places simultaneously, and those records can diverge. Here is why reconciliation is non-negotiable:

- Financial Accuracy: Without reconciliation, your balance sheet could show money you do not actually have.
- Fraud Detection: Discrepancies can indicate unauthorized transactions, employee fraud, or external theft.
- Regulatory Compliance: RBI, SEBI, PCI-DSS, SOX, and other regulations legally require accurate financial records.
- Customer Trust: If customer payments are recorded wrongly, disputes arise — destroying trust.
- Audit Readiness: Auditors need a clean trail. Unreconciled books can trigger regulatory action.
- Cash Flow Management: Knowing your real cash position requires reconciled records.

1.3 What Business Problems Does It Solve?

Problem	Without Reconciliation	With Reconciliation
Duplicate Payments	Pay vendor twice, lose money	Detected and flagged before payment
Missing Settlements	Gateway collected but did not transfer	Caught within 24 hours
Ghost Transactions	Fake records inflate revenue	Mismatch detected immediately
Timing Differences	Month-end books look wrong	Explained and documented
Bank Errors	Bank charges wrong fee	Identified and disputed
System Glitches	Transaction processed twice	Duplicate flagged for review
Human Error	Wrong amount entered	Amount mismatch triggers exception

1.4 What Happens If Reconciliation Is NOT Done?

The consequences of skipping reconciliation compound over time and can be catastrophic:

1. Short Term (Days): Duplicate payments made, vendors paid wrong amounts, customers charged incorrectly.
2. Medium Term (Weeks): Cash shortfalls not detected, payroll errors unnoticed, fraudulent transactions go through.
3. Long Term (Months): Regulatory fines, failed audits, investor loss of confidence, potential criminal liability for CFO/CEO.
4. Catastrophic (Years): Company collapse. Wirecard AG (2020) — a €1.9 billion fraud went undetected partly due to inadequate reconciliation controls.

1.5 Real-World Examples

Banks

A bank like HDFC processes 10 million+ transactions per day. At end of day, their Core Banking System (CBS) must reconcile with: RBI's RTGS/NEFT systems, NPCI's UPI switch, SWIFT network for international transfers, ATM networks (Visa/Mastercard), and internal nostro/vostro accounts. Even a 0.001% error rate = 10,000 wrong transactions per day.

FinTechs (e.g., Razorpay, PayU)

A payment gateway like Razorpay sits between merchants and banks. They must reconcile: money collected from customers via their platform, settlement amounts sent to merchants, fees deducted, refunds processed, and failed transactions reversed. If a customer's payment succeeded but the gateway's ledger shows failed — that is a reconciliation break.

E-Commerce (e.g., Flipkart, Amazon)

When you buy on Flipkart: your bank debits ₹2,000, payment gateway receives ₹2,000, marketplace ledger credits ₹2,000, seller's account gets ₹1,800 (after commission), and Flipkart keeps ₹200. All 5 of these records must reconcile perfectly. Millions of such chains happen daily.

Insurance (e.g., LIC, HDFC Life)

Premium collections, claims payouts, agent commissions, reinsurance premiums — all require reconciliation across multiple systems including the policy management system, payment gateway, bank, and regulator (IRDAI) reporting.

ERP & Accounting (SAP, Oracle Financials)

In ERP systems, every financial transaction creates journal entries in multiple sub-ledgers. The AP (Accounts Payable) sub-ledger must reconcile to the GL (General Ledger). The GL must reconcile to the bank statement. This is called 'sub-ledger to ledger reconciliation.'

PART 2 — Accounting Basics (The Foundation)

2.1 The Double-Entry Principle

The Golden Rule of Accounting

Every financial transaction affects at least TWO accounts. For every debit, there must be an equal credit. This is the foundation of all modern accounting, invented by Luca Pacioli in 1494. This is WHY reconciliation is even possible — every transaction has two sides to compare.

2.2 Debit vs Credit — Demystified

This confuses everyone. Here is the truth: Debit simply means 'left side of the ledger' and Credit means 'right side.' Whether a debit increases or decreases depends on the account type:

Account Type	Increases With	Decreases With	Example
Asset	Debit	Credit	Cash, Bank Balance, Receivables
Liability	Credit	Debit	Loans, Payables, Overdraft
Equity/Capital	Credit	Debit	Owner's Investment
Revenue/Income	Credit	Debit	Sales, Interest Income
Expense	Debit	Credit	Salaries, Rent, Bank Charges

2.3 Journal Entries — Recording Transactions

A Journal Entry is the first place a financial transaction is recorded. It documents what happened, which accounts are affected, and by how much. Format: Date | Account Name | Debit Amount | Credit Amount | Narration.

Example 1: Company receives ₹10,000 payment from customer

Date	Account	Debit (₹)	Credit (₹)	Narration
12 May 2025	Bank Account (Asset)	10,000		Being payment received
12 May 2025	Accounts Receivable (Asset)		10,000	From ABC Traders Pvt Ltd

Explanation: Bank balance goes UP (Debit asset), Receivable goes DOWN (Credit asset — we no longer need to collect).

Example 2: Company pays ₹5,000 to vendor

Date	Account	Debit (₹)	Credit (₹)	Narration
12 May 2025	Accounts Payable (Liability)	5,250		Being vendor payment
12 May 2025	Bank Account (Asset)		5,250	Payment to XYZ Supplies

2.4 The Ledger — The Master Record

A Ledger is a book (or database table) that contains all transactions for a specific account, showing the running balance. Think of it as an account statement for every account.

Sample Bank Account Ledger

Date	Description	Debit (₹)	Credit (₹)	Balance (₹)
01 May 25	Opening Balance			50,000
12 May 25	Received from ABC Traders	10,000		60,000
12 May 25	Paid to XYZ Supplies		5,250	54,750
13 May 25	Sales Receipt	8,000		62,750
14 May 25	Rent Payment		15,000	47,750

2.5 General Ledger (GL)

The General Ledger is the master ledger containing ALL accounts of a business. It is the single source of truth for financial reporting. Every sub-system (sales, purchases, payroll, banking) eventually posts to the GL. The GL is what accountants use to prepare financial statements.

Structure: GL = Chart of Accounts + all journal entries + running balances for every account.

2.6 Trial Balance

A Trial Balance is a list of all GL account balances at a point in time. The sum of all Debit balances must equal the sum of all Credit balances. If they do not, there is an error in the bookkeeping.

Sample Trial Balance (19 May 2025)

Account Name	Debit (₹)	Credit (₹)
Bank Account	47,750	
Accounts Receivable	25,000	
Inventory	30,000	
Accounts Payable		12,500
Sales Revenue		80,000
Salaries Expense	15,000	
Rent Expense	15,000	
Owner's Capital		40,250
TOTAL	₹1,32,750	₹1,32,750

2.7 Accounts Receivable (AR) vs Accounts Payable (AP)

Aspect	Accounts Receivable (AR)	Accounts Payable (AP)
Definition	Money OWED TO the company	Money the company OWES TO others
Asset/Liability	Asset (you will receive cash)	Liability (you owe cash)
Arises from	Selling goods/services on credit	Buying goods/services on credit
Process	Invoice → Follow-up → Collect	Receive invoice → Approve → Pay
Reconciled with	Customer statement / Bank receipts	Vendor statement / Bank payments
Example	Customer bought ₹10k, not yet paid	You bought stock ₹5k, not yet paid

2.8 Cash Book vs Bank Statement

The Cash Book is maintained by the company — it records every receipt and payment the company believes happened through the bank. The Bank Statement is sent by the bank — it shows what the bank recorded on their side. These MUST match after reconciliation. Differences before reconciliation are called 'reconciling items.'

Reconciling Item	Cause	Resolution
Outstanding Cheque	Company issued cheque, bank not yet cleared	Wait for clearing or investigate
Deposits in Transit	Company deposited, not yet showing in bank	Wait 1-2 business days
Bank Charges	Bank deducted fee not in cash book	Update company records
Bank Interest	Bank credited interest not in cash book	Update company records
Errors	Either party made a mistake	Identify, correct, dispute if needed

PART 3 — Banking & Payment Concepts

3.1 Transaction Reference Numbers & UTR

Term	Full Form	Who Issues	Example	Use in Reconciliation
TXN ID	Transaction ID	Internal System	TXN12345	Internal reference
UTR	Unique Transaction Reference	RBI/Bank	HDFC0251234567890	Bank-to-bank tracing
RRN	Retrieval Reference Number	Card Networks	250519123456	Card transaction lookup
ARN	Acquirer Reference Number	Acquiring Bank	74039119051901234567890	Chargeback tracking
VAN/UAN	Virtual Account Number	Payment Gateway	PAY9823456	Automated reconciliation
Ref No	Reference Number	Company / ERP	REF-88421	Matching key in reconciliation

UTR Explained

UTR (Unique Transaction Reference) is a 22-character code assigned by the originating bank for every NEFT/RTGS transfer. It is the most reliable matching key in reconciliation because it is globally unique and immutable. Format: BANKCODEXXX + YYYYMMDD + 10-digit sequence. Example: HDFC0251234567890 → HDFC bank, 02nd branch, 25 May 2025, sequence 1234567890.

3.2 Payment Lifecycle — Complete Flow

When a customer pays ₹1,000 on an e-commerce platform, here is what happens step by step:

Step	Actor	Action	System	Record Created
1	Customer	Clicks Pay	Browser/App	Order record
2	Platform	Creates payment request	Platform backend	Payment intent
3	Gateway (Razorpay)	Tokenizes, routes to bank	Payment gateway	Gateway transaction
4	Customer's Bank	Verifies & debits account	Core Banking System	Debit entry in bank
5	Issuing Bank	Sends authorization	Card network / UPI	Authorization code
6	Gateway	Receives confirmation	Gateway system	Success record
7	Platform	Updates order to paid	Platform backend	Payment confirmation
8	Settlement	Gateway batches + transfers	Gateway settlement	Settlement batch
9	Merchant's Bank	Credits merchant account	Bank CBS	Credit in bank statement

10	Reconciliation	Compare all 3 records	PayRecon System	Match result

3.3 Payment Modes & Settlement Timelines

Payment Mode	Settlement Time	UTR/Reference	Common Issues
NEFT	2 hours (batch)	UTR Number	Late settlement, wrong IFSC
RTGS	30 minutes (real-time)	UTR Number	Minimum ₹2 lakh, timing
IMPS	Instant (24x7)	RRN Number	Limit ₹5 lakh, bank downtime
UPI	Instant (24x7)	UPI Ref ID	Pending states, timeout
Credit/Debit Card	T+2 days	ARN Number	Chargebacks, holds
Net Banking	T+1 day	Bank Ref No	Page timeout, duplicate
Wallet	Instant	Wallet Txn ID	KYC limits, partial balance
Cheque	T+3 days (clearing)	Cheque Number	Bounces, stop payments

3.4 Failed Transactions, Reversals & Chargebacks

Failed Transaction

A payment attempt that did not complete. The customer's account may or may not have been debited. Three scenarios: (A) Never debited — simple case, just mark failed. (B) Debited but not credited to merchant — auto-reversal initiated by bank within 5 days. (C) Both sides unclear — requires manual investigation with bank using UTR.

Reversal

A reversal is initiated by the bank to undo a transaction that was processed in error. It is bank-initiated. In reconciliation, a reversal appears as a new credit entry on the bank statement, referencing the original transaction UTR.

Refund

A refund is initiated by the merchant to return money to the customer. It is merchant-initiated. Refunds create new entries in both the gateway and the bank statement. In reconciliation, unmatched refunds are a common exception.

Chargeback

A chargeback is when a customer disputes a transaction with their bank. The bank forcibly retrieves the money from the merchant. This bypasses the merchant and creates a debit in the merchant's bank account without any corresponding record in their system — a classic reconciliation break.

PART 4 — The Reconciliation Domain

4.1 Types of Reconciliation

Bank Reconciliation

Attribute	Detail
Purpose	Match company's cashbook with bank statement
Inputs	Internal cash book / GL bank account + Bank statement CSV
Outputs	Reconciled statement, list of differences, adjusted balance
Frequency	Daily, weekly, monthly
Challenges	Timing differences, bank charges, returned cheques
Who Does It	Accountant / Finance team

Payment Gateway Reconciliation

Attribute	Detail
Purpose	Match orders in platform vs transactions in gateway vs settlements in bank
Inputs	Platform orders CSV + Gateway report CSV + Bank settlement statement
Outputs	Matched transactions, settlement discrepancies, missing settlements
Frequency	Daily (next-day settlement reconciliation)
Challenges	MDR deductions, failed txn reversals, partial settlements, currency conversion
Who Does It	FinTech ops team, automated systems like PayRecon

Merchant Reconciliation

Done by marketplace platforms (Amazon, Flipkart). Reconciles sales from thousands of sellers, commissions deducted, returns processed, and final settlement to seller. Extremely complex: one marketplace order can have 10+ financial events.

Vendor/Accounts Payable Reconciliation

Company compares its Accounts Payable ledger against vendor-provided statements. Identifies duplicate invoices, payments made but not acknowledged by vendor, or invoices received but not processed.

PART 5 — Architecture Analysis (Image 2 Deep Dive)

5.1 Overview of the 6-Stage Flow

The architecture diagram shows a linear, auditable workflow with 6 stages. Each stage has a clear status, inputs, outputs, and a database interaction. All stages feed into a central MongoDB database.

5.2 Stage 1 — Upload Files

Attribute	Detail
Purpose	Ingest raw data from two independent sources into the system
Actor	Maker (finance staff with upload permission)
Inputs	Internal Ledger CSV, Bank Statement CSV (drag-drop or browse)
Processing	File validation (size, format, headers), parsing, duplicate detection, batch creation
Outputs	ReconciliationBatch record in DB, parsed transaction rows stored in two collections
Status Set	Uploaded
DB Tables	reconciliation_batches, ledger_transactions, bank_transactions
Business Rules	File must be CSV, max 50MB, required columns must exist, no empty files
Error Handling	Invalid format → reject with error message, partial rows → flag for review

From UI Image: The Upload screen shows two separate drop zones — Internal Ledger CSV (left, green icon) and Bank Statement CSV (right, blue icon). Batch metadata (name, date, description) is captured before processing. The 'Upload & Process Files' button triggers the entire pipeline.

5.3 Stage 2 — Auto Matching Engine

Attribute	Detail
Purpose	Automatically pair ledger transactions with bank statement transactions
Actor	System (automated, no human involvement)
Inputs	All parsed ledger rows and bank rows for the batch
Processing	Multi-pass matching: exact match → fuzzy match → partial match → no match
Matching Keys	Amount (exact), Date (within tolerance), Reference Number, UTR, Description (fuzzy)
Outputs	match_results collection with confidence score per pair
Status Set	Matching Completed
DB Tables	match_results (stores pairs + confidence score + match type)
AI Integration	ML model assigns confidence; NLP parses description fields for fuzzy matching
Performance	Must process 50,000+ rows in <60 seconds; uses parallel processing

5.4 Stage 3 — Review & Flags

After matching, the system categorizes every transaction into one of the flag types. These flags are what the Maker reviews.

Flag Type	Definition	Action Required
Matched	Ledger and bank record pair perfectly (high confidence)	No action needed — auto-approved
Unmatched	Ledger record has NO corresponding bank record	Maker must investigate
Duplicate	Same transaction appears more than once	Maker must mark which is real
Amount Mismatch	Records found but amounts differ	Maker must reconcile difference
Date Mismatch	Same txn but on different dates (timing difference)	Maker must confirm or dispute

From UI Image: The reconciliation batch screen shows tabs: All (12,540) | Matched (11,450) | Partial Match (60) | Unmatched (750) | Duplicates (230) | Mismatch (70). The side-by-side view shows Ledger (Internal) on the left and Bank Statement on the right with color-coded comparison.

5.5 Stage 4 — Maker Review & Actions

Attribute	Detail
Purpose	Provide a human review layer for all flagged/unmatched transactions
Actor	Maker — a finance/ops staff member with review permissions
Available Actions	Approve Match, Reject Match, Manual Override (with mandatory reason), Escalate
Override Process	Maker selects mismatched pair → enters override reason → system logs → moves to checker
Constraints	Maker CANNOT approve their own submissions (segregation of duties)
Status Set	Submitted to Checker
DB Tables	maker_actions (stores every action with timestamp, reason, IP address)
Audit Logged	Every click, every reason entered, every status change

5.6 Stage 5 — Checker Review & Approval

Attribute	Detail
Purpose	Provide a second independent review (four-eyes principle)
Actor	Checker — senior staff or manager with approval authority
Available Actions	Approve Batch, Send Back for Changes (with reason), Partial Approve
Key Verification	Checker reviews all Maker overrides, checks for pattern anomalies, spot-checks matches
Status Set	Final Approved (or Sent Back)
DB Tables	checker_actions (stores decision, comments, timestamp)
Business Rule	Checker CANNOT approve if they were involved as Maker in the same batch
Escalation	Unusual mismatches → escalate to CFO / Compliance team

5.7 Stage 6 — Audit Log & Final Report

Once the Checker approves, the system: locks the batch (immutable), generates the reconciliation report (PDF/Excel), writes a final audit log entry, sends notifications, and updates the dashboard.

From UI Image: The Reports screen shows a Reconciliation Summary Report with batch details, a donut chart of matched vs unmatched, and a summary table. Format choices include PDF. 'Generate Report' button triggers async report generation.

PART 6 — Transaction Matching Engine (Deep Dive)

6.1 Matching Strategy Overview

The matching engine runs multiple strategies in sequence. If a transaction is matched by an earlier strategy, it is not re-processed by later ones. This 'waterfall' approach maximizes performance and confidence.

Pass	Strategy	Confidence	Speed	Volume
Pass 1	Exact Match (amount + date + UTR)	99-100%	Fastest	~85% of transactions
Pass 2	Exact Match (amount + date + ref no)	95-99%	Fast	~8% of remaining
Pass 3	Fuzzy Match (amount + date ±2 days + description NLP)	70-94%	Medium	~4% of remaining
Pass 4	Partial / One-to-Many matching	50-69%	Slow	~2% of remaining
Pass 5	Manual / Unmatched	N/A	N/A	~1% — flagged for human review

6.2 Match Types Explained

Exact Match

All key fields match perfectly: Amount = ₹10,000.00, Date = 12 May 2025, UTR = HDFC0251234567890. Confidence Score: 100%. No human review needed.

Fuzzy Match

Used when description fields are similar but not identical. Algorithm: Levenshtein distance or Jaro-Winkler similarity. Example: Ledger has 'ABC Traders Pvt Ltd' and Bank has 'ABC TRADERS PVT. LTD'. Similarity score: 94%. Amount exact, date exact → overall confidence 92%.

One-to-Many Matching

One ledger transaction of ₹15,000 matches TWO bank transactions: ₹10,000 + ₹5,000. This happens when a payment was split across two banking sessions. The system detects that the sum matches and proposes a composite match. Common in batch payment processing.

Many-to-One Matching

Multiple ledger entries correspond to a single consolidated bank entry. Example: Three individual invoices (₹3,000 + ₹4,000 + ₹3,000 = ₹10,000) were paid together by the payer as one ₹10,000 bank transfer.

Many-to-Many Matching

Most complex. Multiple ledger entries match multiple bank entries. Requires solving an optimization problem to find the best possible grouping. Usually limited to known counterparties with pre-agreed payment terms.

6.3 Matching Criteria Details

Criterion	Match Logic	Tolerance	Weight in Score
Amount	Exact numeric match after currency normalization	0 (no tolerance for exact pass)	40%

Date	Calendar date comparison	±0 days (exact), ±2 days (fuzzy)	20%
UTR Number	String exact match	None — must be exact	25%
Reference Number	String match after normalization (remove spaces, case)	None or prefix match	20%
Account Number	Last 4 digits or full match	None	10%
Description	NLP similarity (TF-IDF / Jaro-Winkler)	>85% similarity	5%

PART 7 — Confidence Score System

7.1 What Is a Confidence Score?

A Confidence Score is a number between 0 and 100 (or 0.0 to 1.0) that represents how certain the system is that a proposed match between a ledger transaction and a bank transaction is correct. Higher score = more certain = less human review needed.

Why Not Just Say Match/No Match?

Because reality is not binary. A transaction might have amount matched exactly but the reference number differs by one character. Is it a match? Probably yes, but not certainly. The confidence score quantifies this uncertainty so the system can route it appropriately — auto-approve high confidence, human-review low confidence.

7.2 Confidence Score Formula

Weighted scoring model where each matching criterion contributes a weighted partial score:

```
Confidence_Score = (W_amount × S_amount) + (W_date × S_date) +
                   (W_utr × S_utr) + (W_ref × S_ref) + (W_desc × S_desc)

Where:
  W = Weight (must sum to 1.0)
  S = Score for that criterion (0 to 1)

Default Weights:
  W_amount = 0.40  (40% — amount must match)
  W_utr    = 0.25  (25% — UTR is very reliable)
  W_date   = 0.15  (15% — dates can differ slightly)
  W_ref    = 0.15  (15% — reference numbers help)
  W_desc   = 0.05  (5%  — descriptions often differ)

Example Calculation:
  Ledger: ₹10,000 | 12-May | UTR: HDFC001 | Ref: TXN123
  Bank:   ₹10,000 | 13-May | UTR: HDFC001 | Ref: TXN-123

  S_amount = 1.00 (exact match)
  S_utr    = 1.00 (exact match)
  S_date   = 0.70 (1 day difference → penalized)
  S_ref    = 0.90 (TXN123 vs TXN-123 → 90% similar)
  S_desc   = 0.80 (similar after normalization)

  Score = (0.40×1.00) + (0.25×1.00) + (0.15×0.70) + (0.15×0.90) + (0.05×0.80)
        = 0.40 + 0.25 + 0.105 + 0.135 + 0.04
        = 0.93 → 93% Confidence → Auto-approve threshold
```

7.3 Threshold Configuration

Confidence Range	Classification	System Action	Human Review

95 – 100%	High Confidence Match	Auto-matched, no review needed	No
80 – 94%	Good Match	Matched, but flagged for spot-check	Optional
60 – 79%	Partial Match	Proposed match, Maker must confirm	Yes — Maker
40 – 59%	Low Confidence	Flagged as uncertain match	Yes — Maker + Checker
0 – 39%	No Match	Unmatched, requires full investigation	Yes — mandatory

PART 8 — Exception Management

8.1 Exception Types & Resolution

Unmatched Transaction

Attribute	Detail
Cause	Transaction exists in one source but not the other
Detection	No match found after all matching passes
Resolution	Investigate with bank, check for pending settlement, create manual journal entry
UI Behavior	Red row with 'Unmatched' badge, 'Investigate' action button

Duplicate Transaction

Attribute	Detail
Cause	Same transaction recorded twice (system glitch, double-submission, retry without idempotency)
Detection	Two records with identical amount, date, and reference number in same source
Resolution	Identify the original, void the duplicate, update records
UI Behavior	Orange badge 'Duplicate', side-by-side display, 'Mark as Duplicate' button

Amount Mismatch

Attribute	Detail
Cause	Bank deducted charges (MDR, GST), currency conversion, rounding errors, partial payment
Detection	Matching pair found but amounts differ by >0.01
Resolution	Check for MDR/fee, accept difference if within tolerance, raise dispute if unexplained
UI Behavior	Amounts highlighted in red, difference amount shown, 'Justify Difference' field

Date Mismatch

Attribute	Detail
Cause	Timing differences: cheque clearing delay, weekend settlement, timezone differences
Detection	Matching pair found but dates differ by >0 days
Resolution	Accept if within normal settlement window, investigate if >5 business days
UI Behavior	Dates shown in different colors, tooltip explains expected settlement window

Missing Transaction

Attribute	Detail

Cause	Transaction was processed but never recorded in internal system
Detection	Bank statement has entry, internal ledger has nothing
Resolution	Create ledger entry, investigate WHY it was not recorded, fix system if needed
UI Behavior	Empty cell on ledger side, bank entry highlighted, 'Create Ledger Entry' button

PART 9 — Maker-Checker Workflow (Four-Eyes Principle)

9.1 What Is Maker-Checker?

Maker-Checker is a dual-control mechanism used in banking and finance where every significant action requires two independent people to complete it: one person initiates (Maker) and a different person approves (Checker). No single person can complete a high-value operation alone.

Real-World Analogy

Think of opening a bank safe: one key is held by the manager, another by the security officer. Neither can open it alone. In PayRecon: the Maker reviews and submits reconciliation; the Checker must independently verify and approve. Neither can complete the cycle alone.

9.2 Why Enterprises Use It

- Prevents Fraud: No single employee can manipulate records and approve them undetected
- Segregation of Duties: Accounting principle requiring different people for record and approval
- Error Catching: Second pair of eyes catches mistakes the first person missed
- Regulatory Requirement: RBI, SEBI, and internal audit standards mandate dual control for financial transactions above defined thresholds
- Audit Trail: Creates a verifiable paper trail showing independent review

9.3 Complete Workflow State Machine

State	Who	Available Actions	Next State
Uploaded	System	Auto-trigger matching	Matching Completed
Matching Completed	System	Generate flags	Pending Review
Pending Review	Maker	Review items, take actions	In Review
In Review	Maker	Approve matches, override, escalate, submit	Submitted to Checker
Submitted to Checker	Checker	Review Maker actions, spot-check	Under Checker Review
Under Checker Review	Checker	Approve, Send Back, Partially Approve	Final Approved / Sent Back
Sent Back	Maker	Review checker comments, rework, resubmit	Submitted to Checker
Final Approved	System	Lock batch, generate report, audit log	Reconciliation Complete
Reconciliation Complete	System/Auditor	View only — no edits	Archived

9.4 Constraints & Business Rules

- A Maker cannot approve their own submission (system enforces by user ID check)
- A Checker cannot have been involved in the batch as a Maker
- Maker and Checker must have different usernames AND different roles
- 'Send Back' requires a mandatory reason (minimum 20 characters)
- An approved batch CANNOT be re-opened without Admin + Audit log entry
- All overrides require a reason that is permanently logged

PART 10 — Audit Trail & Compliance

10.1 What Is an Audit Trail?

An Audit Trail is an immutable, chronological record of every action taken in the system. 'Immutable' means no one — not even system administrators — can delete or modify audit records. It is the system's memory of everything that ever happened.

Think of it like CCTV footage for your application. The CCTV records everything — even if someone deletes a file, the audit log records who deleted it, when, and from which IP address.

10.2 What Must Be Logged

Event Category	Events to Log
Authentication	Login success, Login failure, Password change, Session timeout, MFA events
File Events	Upload started, Upload completed, File validation error, File deleted
Matching Events	Matching engine started, Matching completed, Confidence scores assigned
Maker Actions	Opened batch, Reviewed item, Approved match, Rejected match, Override applied, Submitted batch
Checker Actions	Opened batch, Reviewed Maker actions, Approved batch, Sent back, Comments added
Admin Events	User created, Role changed, Permission granted/revoked, Settings changed
Data Changes	Any field changed: old value, new value, who changed it, when
Report Events	Report generated, Report downloaded, Report emailed
System Events	Service started/stopped, Error events, Integration calls

10.3 Audit Record Structure

```
// Sample Audit Log Record (MongoDB document)
{
  _id: ObjectId('683abc123def456789'),
  timestamp: ISODate('2025-05-19T10:45:23.456Z'),
  user: {
    id: 'USR001',
    name: 'Rudra',
    role: 'Maker',
    email: 'rudra@company.com'
  },
  action: 'OVERRIDE_APPLIED',
  category: 'MAKER_ACTION',
  entityType: 'MATCH_RESULT',
  entityId: 'MATCH-88421',
  batchId: 'BATCH-250519-001',
  changes: {
    field: 'status',
    oldValue: 'AMOUNT_MISMATCH',
    newValue: 'MANUALLY_MATCHED',
    reason: 'Bank deducted MDR of ₹50. Difference of ₹50 within approved
tolerance.'
```

```
  },
  ipAddress: '192.168.1.101',
  userAgent: 'Chrome/124.0 Windows',
  sessionId: 'sess_abc123xyz',
  immutable: true
}
```

10.4 Immutability Implementation

- Write-only MongoDB collection: application user has INSERT-only access, no UPDATE/DELETE
- Append-only database design: add new correction records instead of editing old ones
- Cryptographic hashing: each audit record stores SHA-256 hash of its content; any tampering breaks the hash
- Audit log backup: real-time replication to separate immutable storage (AWS S3 with Object Lock)
- Periodic hash chain: each record includes hash of previous record, creating a blockchain-like chain

PART 11 — Database Design (MongoDB)

11.1 Entity Relationship Overview

Collection	Purpose	Key Fields
users	System users with auth info	_id, email, passwordHash, role, status
roles	Role definitions	_id, name, permissions[]
reconciliation_batches	Each upload session	_id, batchName, status, makerId, checkerId
ledger_transactions	Internal ledger rows	_id, batchId, txnId, date, amount, reference
bank_transactions	Bank statement rows	_id, batchId, txnDate, credit, debit, utr
match_results	Pairing outcomes	_id, batchId, ledgerTxnId, bankTxnId, matchType, confidence
exceptions	Flagged issues	_id, batchId, exceptionType, status, resolution
maker_actions	All Maker activities	_id, batchId, userId, action, reason, timestamp
checker_actions	All Checker activities	_id, batchId, userId, decision, comments, timestamp
audit_logs	Immutable event log	_id, timestamp, userId, action, changes, ipAddress
reports	Generated reports	_id, batchId, reportType, filePath, generatedAt

11.2 Core Collection Schemas

reconciliation_batches

```
{
  _id: ObjectId,
  batchId: 'BATCH-250519-001',  // Human-readable unique ID
  batchName: 'Reconciliation_19_May_2025',
  reconciliationDate: ISODate('2025-05-19'),
  description: 'Daily reconciliation for 19 May 2025',
  status: 'IN_REVIEW',  // UPLOADED|MATCHING|PENDING_REVIEW|IN_REVIEW|
                        // SUBMITTED|APPROVED|SENT_BACK|COMPLETE
  files: {
    ledger: { name: 'internal_ledger_19052025.csv', size: 2457600, rows: 6250 },
    bank:   { name: 'bank_statement_19052025.csv', size: 2293760, rows: 6248 }
  },
  stats: {
    totalLedger: 6250, totalBank: 6248,
    matched: 5781, unmatched: 469, duplicates: 143, mismatches: 97
  },
  maker: { userId: 'USR001', submittedAt: ISODate },
  checker: { userId: 'USR002', reviewedAt: ISODate, decision: 'APPROVED' },
  createdAt: ISODate, updatedAt: ISODate
}
```

match_results

```
{
  _id: ObjectId,
  matchId: 'MATCH-88421',
  batchId: 'BATCH-250519-001',
  matchType: 'EXACT',  // EXACT|FUZZY|PARTIAL|MANUAL|ONE_TO_MANY|MANY_TO_ONE
  confidenceScore: 98.5,
  status: 'MATCHED',  // MATCHED|UNMATCHED|MISMATCH|DUPLICATE|OVERRIDE
  ledgerTransaction: {
    txnId: 'TXN12345',
    date: ISODate('2025-05-12'),
    description: 'ABC Traders Pvt Ltd',
    amount: 10000.00
  },
  bankTransaction: {
    txnId: 'TXN12345',
    date: ISODate('2025-05-13'),
    description: 'ABC TRADERS PVT LTD',
    amount: 9850.00,
    utr: 'HDFC0251234567890'
  },
  difference: { amount: 150.00, days: 1 },
  flags: ['AMOUNT_MISMATCH', 'DATE_MISMATCH'],
  override: { applied: true, reason: 'MDR deduction ₹150', userId: 'USR001', at:
ISODate },
  createdAt: ISODate, updatedAt: ISODate
}
```

PART 12 — Backend Architecture

12.1 Service Architecture

Service	Responsibility	Tech Stack	Key APIs
API Gateway	Route, auth validate, rate limit all incoming requests	Node.js + Express / Nginx	All endpoints pass through
Auth Service	JWT issue, refresh, validate; user management	Node.js, bcrypt, jsonwebtoken	/auth/login, /auth/refresh
File Service	Receive uploads, validate, parse CSV, store	Node.js, multer, csv-parser	/files/upload, /files/validate
Matching Engine	Run matching algorithms, assign confidence scores	Node.js / Python worker	/match/run, /match/status
Reconciliation Service	Batch management, status transitions, exception mgmt	Node.js	/batches, /exceptions
Approval Service	Maker/Checker workflow, state machine enforcement	Node.js	/maker, /checker, /approvals
Audit Service	Write-only audit log; never reads, never deletes	Node.js, separate DB user	/audit/log (internal only)
Report Service	PDF/Excel generation async	Node.js, PDFKit/ExcelJS	/reports/generate, /reports/download
Notification Service	Email/SMS/in-app alerts	Node.js, Nodemailer/AWS SES	Internal pub/sub only
AI Service	Fuzzy matching, anomaly detection, NLP queries	Python, FastAPI, scikit-learn	/ai/match, /ai/explain, /ai/query

12.2 Technology Stack

Layer	Technology	Purpose
Frontend	React.js + TypeScript + TailwindCSS	UI as shown in the screenshots
Backend API	Node.js + Express.js	REST API server
Database	MongoDB + Mongoose	Primary data store (as per architecture)
Cache	Redis	Session store, rate limiting, job queue status
Job Queue	Bull (Redis-backed)	Async processing: matching, reports
File Storage	AWS S3 / Local MinIO	Store uploaded CSV files
AI/ML	Python + FastAPI + scikit-learn	Matching ML model, NLP
Authentication	JWT (RS256) + bcrypt	Stateless auth
PDF Generation	PDFKit / Puppeteer	Report generation
Email	Nodemailer + AWS SES	Notifications
Logging	Winston + ELK Stack	Application logs
Monitoring	Prometheus + Grafana	Performance monitoring

PART 13 — REST API Design

13.1 Authentication APIs

```
POST /api/v1/auth/login
Request:  { email: 'rudra@company.com', password: 'SecurePass@123' }
Response: { accessToken: 'eyJ...', refreshToken: 'eyJ...', user: { id, name, role
} }
Errors:   401 (invalid creds), 423 (account locked), 429 (rate limited)

POST /api/v1/auth/refresh
Request:  { refreshToken: 'eyJ...' }
Response: { accessToken: 'eyJ...' }

POST /api/v1/auth/logout
Headers:  Authorization: Bearer <token>
Response: { message: 'Logged out successfully' }
```

13.2 Upload & Batch APIs

```
POST /api/v1/batches/upload
Headers:  Authorization: Bearer <token>, Content-Type: multipart/form-data
Request:  FormData { ledgerFile, bankFile, batchName, reconciliationDate,
description }
Response: { batchId: 'BATCH-250519-001', status: 'UPLOADED', stats: { ledgerRows,
bankRows } }
Validation: Both files required, CSV format, max 50MB each, headers validated

GET /api/v1/batches
Query:    page=1&limit=10&status=IN_REVIEW&dateFrom=2025-05-01
Response: { batches: [...], total: 45, page: 1, limit: 10 }

GET /api/v1/batches/:batchId
Response: Full batch object with stats, maker/checker info, file details
```

13.3 Reconciliation & Matching APIs

```
POST /api/v1/batches/:batchId/run-matching
Response: { jobId: 'JOB-123', status: 'QUEUED' }  // Async job

GET /api/v1/batches/:batchId/transactions
Query:    type=unmatched&page=1&limit=50&search=ABC+Traders
Response: { transactions: [{ ledger: {...}, bank: {...}, match: {...} }], total:
750 }

GET /api/v1/batches/:batchId/summary
Response: { total: 12540, matched: 11450, matchRate: 91.3, unmatched: 750, ... }
```

13.4 Maker & Checker APIs

```
POST /api/v1/maker/actions
Body:     { batchId, matchId, action: 'OVERRIDE', reason: 'MDR deduction...',
newStatus: 'MATCHED' }
Response: { actionId: 'ACT-001', status: 'RECORDED', auditId: 'AUD-789' }

POST /api/v1/maker/batches/:batchId/submit
Body:     { notes: 'All exceptions reviewed. 3 overrides applied.' }
Response: { batchId, status: 'SUBMITTED_TO_CHECKER', submittedAt: ISODate }

POST /api/v1/checker/batches/:batchId/approve
Body:     { decision: 'APPROVED', comments: 'Reviewed all overrides. Acceptable.'
}
Response: { batchId, status: 'FINAL_APPROVED', approvedAt: ISODate }

POST /api/v1/checker/batches/:batchId/send-back
Body:     { reason: 'Override on TXN88421 needs better justification. Provide bank
receipt.' }
Response: { batchId, status: 'SENT_BACK', sentBackAt: ISODate }
```

PART 14 — Security Architecture

14.1 JWT Authentication

JSON Web Tokens (JWT) are used for stateless authentication. The server issues a signed token at login; every subsequent request includes this token in the Authorization header. The server validates the signature without hitting the database.

JWT Component	Content	Security Note
Header	Algorithm (RS256), Token type	Use RS256 (asymmetric), never HS256 in production
Payload (Claims)	userId, role, permissions, exp, iat, jti	Never store sensitive data in payload — it is Base64 decodable
Signature	HMAC of header+payload with private key	Use 2048-bit RSA keys, rotate every 90 days
Access Token TTL	15 minutes	Short-lived to limit exposure
Refresh Token TTL	7 days	Store hashed in DB, allow revocation

14.2 Role-Based Access Control (RBAC)

Permission	Admin	Checker	Maker	Viewer
Upload Files	✓	✗	✓	✗
Run Matching	✓	✗	✓	✗
Review Transactions	✓	✓	✓	✓ (read)
Apply Overrides	✓	✗	✓	✗
Submit to Checker	✓	✗	✓	✗
Approve Batch	✓	✓	✗	✗
Send Back Batch	✓	✓	✗	✗
Generate Reports	✓	✓	✓	✓
View Audit Trail	✓	✓	✗	✗
Manage Users	✓	✗	✗	✗
Delete/Archive Batch	✓	✗	✗	✗

14.3 Security Checklist

- Password Storage: bcrypt with cost factor 12+ (never plain text, never MD5/SHA1)
- File Upload Security: validate MIME type, not just extension; scan with ClamAV; reject files >50MB; store outside web root
- SQL/NoSQL Injection: use parameterized queries; mongoose schema validation; never concatenate user input
- XSS Prevention: sanitize all output; Content-Security-Policy headers; HttpOnly + Secure cookies

- CSRF Protection: SameSite=Strict cookies; CSRF tokens for state-changing operations
- Rate Limiting: 5 login attempts per minute per IP; 100 API calls per minute per user
- Encryption: TLS 1.3 for all connections; AES-256 for sensitive data at rest; key management via AWS KMS
- Audit Log Protection: separate database user with INSERT-only rights on audit_logs collection

PART 15 — Reporting System

15.1 Report Types

Report Name	Audience	Frequency	Content	Format
Reconciliation Summary	Ops Team, Manager	Per batch	Matched %, exceptions count, batch metadata	PDF, Excel
Exception Report	Ops Team	Per batch	Detailed list of every unmatched/mismatch with status	Excel, PDF
Audit Trail Report	Compliance, Auditors	On demand	All actions for a batch or date range with full details	PDF, CSV
Daily Summary	CFO, Finance Head	Daily (automated)	Day's totals: value processed, match rate, pending items	Email, PDF
Monthly Reconciliation	CFO, Board	Monthly	Trend analysis, recurring exceptions, resolution rates	PDF, Excel
Unmatched Items	Investigation Team	Per batch	All unmatched with suggested actions	Excel
Aged Items Report	Compliance	Weekly	Items unmatched >7 days, >30 days, >90 days	PDF, Excel

15.2 Sample Report Structure — Reconciliation Summary

Section	Content
Header	Company logo, Report title, Batch ID, Generated by, Generated at, Date range
Executive Summary	Total transactions, Match rate (91.30%), Unmatched count & value, Status
Statistics Table	Matched: 11,450 (91.30%) | Unmatched: 750 (5.97%) | Duplicates: 230 | Mismatch: 70
Charts	Donut chart (matched vs unmatched), Trend line (daily match rate over period)
Exception Detail	Table of each exception: TXN ID, Amount, Type, Status, Assigned to
Override Summary	List of all manual overrides: Who approved, What reason, Which transactions
Approval Chain	Maker: Rudra, Submitted at: 10:45, Checker: Ananya, Approved at: 11:20
Footer	Page numbers, Confidentiality notice, System version, Digital signature hash

PART 16 — AI & Intelligence Features

16.1 Smart Fuzzy Matching (NLP)

Traditional exact matching fails on description fields because banks and ERP systems format text differently. AI-powered fuzzy matching uses:

- TF-IDF Vectorization: Converts descriptions to numeric vectors; cosine similarity finds best matching pair.
- Jaro-Winkler Distance: String similarity algorithm optimized for short strings like names.
- Named Entity Recognition (NER): Extracts company names, amounts, dates from unstructured text.
- Learned patterns: ML model trained on historical matches learns that 'ABC Traders' = 'ABC TRADERS PVT LTD' = 'A.B.C. Trading Co.'

16.2 Anomaly Detection

Uses Isolation Forest or Autoencoder models to detect unusual patterns:

Anomaly Type	Detection Method	Action
Unusually large amount	Z-score > 3σ from mean for that merchant	Flag for manual review, notify CFO
Transaction at unusual time	Time-series anomaly (e.g., 2 AM transaction)	Flag + alert security team
Sudden volume spike	3x normal daily transaction count	Auto-pause and alert
Duplicate amount patterns	Frequent exact-amount duplicates	Flag as potential system error
Velocity anomaly	Same reference number used multiple times	Block and escalate

16.3 AI Mismatch Explainer

When a mismatch is detected, instead of showing a raw difference, the AI generates a human-readable explanation:

```
Input:
  Ledger: ₹10,000 | ABC Traders | 12-May
  Bank:   ₹9,850  | ABC TRADERS | 13-May

AI Explanation Output:
  'This appears to be the same transaction. The ₹150 difference likely represents
  the payment gateway MDR (Merchant Discount Rate) of 1.5% on ₹10,000.
  The 1-day date difference is within the normal T+1 settlement window for
  NEFT transactions. Suggested action: Accept match with MDR note.'

Confidence: 94%  |  Suggested Action: Accept with MDR explanation
```

16.4 Chat With Data (RAG — Retrieval-Augmented Generation)

Allows finance team to query reconciliation data using natural language:

User Query	System Action	Response

'Show all unmatched transactions above ₹50,000'	Convert to MongoDB query, execute, format	Table of 12 transactions with details
'Why was BATCH-250519-001 sent back?'	Retrieve checker action record, summarize reason	Checker comment + specific items flagged
'What is our average match rate this month?'	Aggregate query across May batches	92.4% average, trend chart
'Who approved the TXN88421 override?'	Query audit_logs for override event	Rudra (Maker) → Ananya (Checker) approved at 11:20

PART 17 — UI Analysis (Screen by Screen)

17.1 Dashboard Screen

Widget	Purpose	Backend API	Permission
KPI Cards (4 metrics)	Total/Matched/Unmatched/Duplicates/Pending count	GET /batches/stats	All roles
Reconciliation % Donut	Visual of match rate	Same stats API	All roles
Transaction Trend Line Chart	Daily matched/unmatched/duplicates over date range	GET /batches/trend	All roles
Recent Batches Table	Last 10 batches with status, actions	GET /batches?limit=10	All roles
Pending Approvals Badge	Count of batches awaiting checker (shown in nav)	GET /approvals/pending	Checker/Admin
Date Range Filter	Filter all metrics by date range	Query param propagation	All roles

17.2 Upload Screen

Two drag-drop zones side by side. Left: Internal Ledger CSV (green icon). Right: Bank Statement CSV (blue icon). Below: Batch Name, Reconciliation Date, Description fields. 'Upload & Process Files' button disabled until both files uploaded and metadata filled. Shows file name and size after selection. Progress bar during upload and parsing.

17.3 Reconciliation Screen (Diff View)

The most complex screen. Shows a split view: Ledger Internal (left) vs Bank Statement (right) with a Comparison column in the middle. Each row pair is color-coded: Green = Matched, Red = Amount/Date Mismatch, Orange = Unmatched, Yellow = Partial Match. Tabs across top filter by match type. Search bar for transaction ID. Export and Columns filter buttons.

17.4 Approvals Screen (My Approvals)

Checker sees all batches awaiting their review. Tabs: Pending (12) | Approved | Rejected | Sent Back. Each row shows Batch ID, Maker name, Upload date, Total transactions, Pending items count, and a 'Review' action button. Clicking Review opens the full reconciliation diff view in 'Checker Mode' where Approve/Send Back buttons are shown.

17.5 Audit Trail Screen

Chronological event log with timestamp, user, role, action type, batch ID, IP address, and Details link. Filterable by date range, user, action type, batch ID. 'Export' button for compliance reporting. Read-only — no edits possible. Color coding: Green = Approvals, Blue = Uploads/Matching, Orange = Overrides, Red = Rejections/Send Backs.

17.6 Reports Screen

Select Batch ID from dropdown, select Report Type (Reconciliation Summary, Exception Report, Audit Trail), select Format (PDF, Excel, CSV). 'Generate Report' triggers async generation. Download link appears when ready. Reports are stored and can be re-downloaded.

PART 18 — Complete User Journey (Simulation)

18.1 Scenario Setup

Company: TechMart Pvt Ltd | Date: 19 May 2025 | Maker: Rudra | Checker: Ananya

Ledger has 6,250 transactions totaling ₹1,23,45,000. Bank statement has 6,248 transactions. Goal: reconcile the differences.

18.2 Step-by-Step Walkthrough

Step	Action	What Happens	API Called	Audit Event
1	Login	Rudra opens PayRecon. Enters email + password. System validates, issues JWT (15 min TTL). Redirected to Dashboard.	POST /auth/login → JWT issued	Audit: LOGIN_SUCCESS logged
2	View Dashboard	Sees 45 batches, 12 pending approvals. Notices yesterday's batch still In Review. Decides to upload today's files.	GET /batches/stats	No audit (read-only)
3	Upload Files	Clicks Upload Files. Drags internal_ledger_19052025.csv (2.45MB, 6,250 rows) and bank_statement_19052025.csv (2.28MB, 6,248 rows). Enters batch name. Clicks Upload.	POST /batches/upload → File validation → CSV parsing	Audit: FILE_UPLOADED × 2
4	Matching Runs	System automatically starts matching engine. 91.3% matched in 45 seconds. Rudra sees status change to Pending Review.	POST /match/run (async job) → Bull queue → Worker process	Audit: MATCHING_COMPLETED
5	Review Exceptions	Rudra opens batch. Sees 750 unmatched, 230 duplicates, 70 mismatches. Starts with Mismatches tab. Finds TXN88421: ledger ₹10,000 vs bank ₹9,850.	GET /batches/BATCH-250519-001/transactions?type=mismatch	No audit (view)
6	Apply Override	For TXN88421: Rudra investigates, sees MDR deduction of ₹150. Enters reason: 'Bank deducted MDR of 1.5%. Difference ₹150 within approved threshold.' Clicks Override.	POST /maker/actions { action: OVERRIDE, reason: '...', matchId }	Audit: OVERRIDE_APPLIED with full reason
7	Submit to Checker	After reviewing all 320 exceptions (approving 310, overriding 8, escalating 2), Rudra clicks Submit. Adds note: '8 overrides applied, 2 escalated to CFO.'	POST /maker/batches/BATCH-250519-001/submit	Audit: BATCH_SUBMITTED

8	Checker Reviews	Ananya receives notification email. Opens My Approvals. Sees batch from Rudra. Reviews each override. Clicks into TXN88421 override — reads reason, checks MDR policy, agrees.	GET /checker/batches/BATCH-250519-001	Audit: CHECKER_REVIEW_STARTED
9	Checker Approves	Ananya satisfied with all 8 overrides. Adds comment: 'All overrides verified against MDR schedule. 2 escalated items noted for CFO.' Clicks Approve.	POST /checker/batches/BATCH-250519-001/approve	Audit: BATCH_APPROVED
10	Report Generated	System auto-generates Reconciliation Summary PDF. Sends daily summary email to Finance Head. Batch status: Reconciliation Complete. Immutable lock applied.	POST /reports/generate (async) → PDF → S3 storage	Audit: REPORT_GENERATED, BATCH_LOCKED

PART 19 — Project Implementation Roadmap

Phase 1 — MVP (Weeks 1-6)

Category	Deliverables
Backend	Node.js + Express setup, MongoDB connection, JWT auth, user CRUD, file upload + CSV parsing
Matching Engine	Exact matching on amount + date + reference, basic confidence scoring, flag generation
Database	users, roles, batches, ledger_txns, bank_txns, match_results collections
Frontend	Login page, Dashboard (basic), Upload screen, Reconciliation list view
APIs	Auth APIs, Upload API, Batch CRUD, Basic matching API
Testing	Unit tests for matching logic, API integration tests with Postman

Phase 2 — Production System (Weeks 7-14)

Category	Deliverables
Maker-Checker	Full workflow state machine, override system, submission/approval/send-back flow
Audit Trail	Complete audit logging service, immutable records, audit viewer UI
Enhanced Matching	Fuzzy matching, one-to-many, date tolerance, confidence score tuning
Reports	PDF report generation, exception report, audit report, email notifications
Security	RBAC enforcement, rate limiting, file validation hardening, HTTPS
UI Completion	Diff view, approval screen, audit trail UI, reports UI, user management
Database	exceptions, maker_actions, checker_actions, audit_logs, reports collections

Phase 3 — Enterprise Features (Weeks 15-22)

Category	Deliverables
Performance	Bull job queues for async processing, Redis caching, database indexing, pagination
Scalability	Docker containerization, Kubernetes deployment, load balancing
Multi-tenancy	Organization-based data isolation, white-labeling
Advanced Reports	Dashboard analytics, trend analysis, aged items report, scheduled reports
Integrations	ERP connector (SAP/Oracle), bank API integration, email/SMS notifications
Compliance	Data retention policies, GDPR compliance, SOC 2 controls, PCI-DSS audit prep

Phase 4 — AI Features (Weeks 23-30)

Category	Deliverables

Smart Matching	ML model for confidence scoring trained on historical data
Fuzzy NLP	TF-IDF + NER for description matching, entity normalization
Anomaly Detection	Isolation Forest model, real-time alerting
AI Explainer	GPT-4 / Claude API integration for mismatch explanations
Chat With Data	RAG pipeline: embeddings → vector DB → natural language queries
Predictive	Predict settlement delays, identify recurring exception patterns

PART 20 — Interview Preparation (Top 100 Q&A)

Section A: FinTech Domain Questions

#	Question	Answer
1	What is payment reconciliation?	The process of comparing internal financial records with external sources (bank/gateway) to verify accuracy, detect discrepancies, and ensure all transactions are accounted for correctly.
2	What is the difference between a reversal and a refund?	Reversal is initiated by the bank/system to undo an erroneous transaction (bank-initiated). Refund is initiated by the merchant to return money to the customer (merchant-initiated). Both create new entries in financial records.
3	What is UTR and why is it important for reconciliation?	Unique Transaction Reference — a 22-character code issued by the originating bank for NEFT/RTGS. It is globally unique and immutable, making it the most reliable matching key. Without UTR, transactions are matched by amount + date + description which is less reliable.
4	What is settlement T+1, T+2?	T=Transaction day. T+1 means settlement happens the next business day. T+2 means 2 business days after. Payment gateways typically settle on T+1 or T+2. This creates date differences in reconciliation that are expected and must be handled.
5	What is a chargeback and how does it affect reconciliation?	A chargeback is a forced transaction reversal initiated by the customer's bank. It creates a debit in the merchant's account without any corresponding merchant-initiated record, causing a reconciliation break. Must be tracked separately.
6	What is MDR and why does it cause reconciliation mismatches?	Merchant Discount Rate — fee charged by payment gateway (typically 1-2.5%). Gateway deducts MDR before settling. So if customer pays ₹10,000, merchant receives ₹9,850 (after 1.5% MDR). Internal ledger shows ₹10,000, bank shows ₹9,850 → amount mismatch that must be explained.
7	What is the difference between NEFT, RTGS, and IMPS?	NEFT: batch settlement every 2 hours, any amount. RTGS: real-time, minimum ₹2 lakh, business hours. IMPS: instant, 24x7, max ₹5 lakh per transaction. Each has different UTR formats, settlement timelines, and error patterns.
8	What is a nostro/vostro account?	Nostro = 'our account held at another bank' (from our perspective). Vostro = 'your account held at our bank' (from the other bank's perspective). Same account, different perspectives. Banks reconcile nostro accounts daily — crucial for international transfers.
9	What is the float in banking?	Float is money in transit — debited from payer but not yet credited to payee. During clearing periods, float exists. Reconciliation must account for float as a timing difference, not an error.
10	What is PCI-DSS?	Payment Card Industry Data Security Standard — security standard for organizations that handle card data. Requires encryption, access control, monitoring, regular audits. Non-compliance = fines + loss of card processing ability.

Section B: Reconciliation Technical Questions

#	Question	Answer
11	How do you handle one-to-many matching?	When one ledger entry maps to multiple bank entries (sum = ledger amount), the engine groups bank entries by candidate set and checks

		if any combination sums to the ledger amount. Uses subset-sum algorithm for small sets. Flag for human review if confidence < 80%.
12	How would you design the confidence score formula?	Weighted sum of individual field match scores: amount (40%), UTR (25%), date (15%), reference (15%), description (5%). Each field gets 0-1 score based on exact match, fuzzy similarity, or tolerance. Threshold: >95% auto-match, 60-95% human review, <60% unmatched.
13	How do you prevent duplicate processing?	Idempotency key: hash of (batchId + txnId + amount). Before processing, check if hash already exists. File-level: hash the entire CSV file; if same hash uploaded again, reject as duplicate batch.
14	How would you handle 10 million transactions?	Batch processing via job queue (Bull). Parallel workers (4-8 instances). Database indexing on batchId + status. Pagination for all API responses. Incremental matching. Cache frequently accessed batch summaries in Redis.
15	What is the two-pass matching strategy?	Pass 1: exact match (fast, high confidence). Pass 2: fuzzy match on unmatched from pass 1 (slower, lower confidence). Only unmatched after pass 2 go to human review. This maximizes auto-match rate while keeping precision high.

Section C: System Design Questions

#	Question	Answer
16	Design the database schema for matching results.	match_results collection: _id, batchId (indexed), ledgerTxnId (ref), bankTxnId (ref), matchType (EXACT/FUZZY/PARTIAL/MANUAL), confidenceScore (number), status (enum), flags (array), difference (object), override (object), auditTrail (array). Compound index on (batchId, status).
17	How do you make the audit log immutable?	Three layers: (1) Database user with INSERT-only privileges on audit collection. (2) Application layer rejects any UPDATE/DELETE on audit records. (3) Cryptographic hash chain: each record stores SHA-256 of its content + hash of previous record. Any tampering breaks the chain.
18	How would you implement the Maker-Checker state machine?	Store status as enum in reconciliation_batches. Create statusTransitions config: { UPLOADED: ['MATCHING'], MATCHING: ['PENDING_REVIEW'], ... }. On every status change, validate: (1) transition is allowed, (2) user has permission for this transition, (3) Maker != Checker. Log transition to audit.
19	How do you handle file upload for large CSV files?	Multipart upload to S3/MinIO. Stream parsing (not load entire file into memory). csv-parser with backpressure. Chunk inserts to MongoDB (500 rows per batch). Bull job for async processing. Return jobId immediately; polling or WebSocket for progress.
20	How do you design notifications?	Event-driven: reconciliation service emits events (batch_submitted, batch_approved, etc.) to a message bus (Bull/Redis). Notification service consumes events, looks up user preferences, sends via appropriate channel (email/SMS/in-app). Template-based messages stored in DB.
21	How would you scale this system to handle 100 companies?	Multi-tenancy: add organizationId to every collection, index on it. Tenant isolation at application layer (middleware adds organizationId to all queries). Separate DB per enterprise tenant (for larger customers). Rate limits per tenant. Tenant-specific configuration (thresholds, matching rules).
22	How do you handle RBAC in the API layer?	JWT contains role claim. Middleware extracts role, checks against permission matrix (defined in code/DB). Granular check at controller level. Example: checkPermission('APPROVE_BATCH') middleware. Permission matrix versioned in DB — can change permissions without code deploy.

23	How do you implement fuzzy matching for descriptions?	Preprocess: lowercase, remove special chars, normalize whitespace. Vectorize using TF-IDF. Compute cosine similarity between ledger and bank description vectors. Threshold: >85% similarity contributes to confidence score. For Indian company names: also check for abbreviation patterns (PVT vs PRIVATE, LTD vs LIMITED).
24	How would you implement the reporting service?	Async job: on report request, create job in Bull queue, return jobId. Worker pulls batch data, generates PDF using PDFKit (or Puppeteer for HTML→PDF). Stores to S3. Updates report record with download URL. Notification service emails link. Cleanup: delete reports >90 days old.
25	How do you test the matching engine?	Unit tests: test each matching strategy in isolation with known inputs and expected outputs. Test edge cases: null amounts, zero amounts, unicode descriptions, very long strings. Integration tests: upload real CSV files, verify match counts. Regression test suite of 1000+ transactions with known expected matches.

Section D: Audit & Compliance Questions

#	Question	Answer
26	What regulations require audit trails?	RBI Master Directions (payment systems), PCI-DSS Requirement 10 (audit log monitoring), SOX Section 404 (internal controls), ISO 27001 (information security), Companies Act 2013 Section 128 (books of account), SEBI LODR (listed companies).
27	What must an audit log capture for every action?	Who (userId, name, role), When (timestamp with timezone, millisecond precision), What (action type, entity type, entity ID), How (IP address, user agent, session ID), Before state (old value), After state (new value), Why (reason if action required one).
28	How long should audit logs be retained?	RBI: 5 years for payment records. PCI-DSS: 1 year online + 3 years archive. SOX: 7 years. Indian IT Act: 3 years. Best practice: retain all audit logs for 7 years (longest requirement), with last 1 year in fast storage and older in cold storage (S3 Glacier).
29	What is segregation of duties and why is it important?	The principle that no single person should have end-to-end control over a financial process. In reconciliation: the person who uploads files (Maker) cannot also approve the reconciliation (Checker). Prevents fraud — a malicious employee would need to collude with at least one other person.
30	What is a SOC 2 audit?	Service Organization Control 2 — an audit framework that evaluates a company's controls around security, availability, processing integrity, confidentiality, and privacy. FinTech companies serving enterprise clients are often required to have SOC 2 Type 2 certification. The reconciliation system must support SOC 2 by providing detailed audit logs and access controls.

BONUS — Missing Modules, Risks & Best Practices

Critical Missing Modules (Not in Architecture but Required)

Missing Module	Why Needed	Priority
Idempotency Layer	Prevent duplicate transaction processing if API called twice	Critical
Data Validation Service	Validate CSV data quality before matching (negative amounts, future dates, malformed IDs)	Critical
Holiday/Weekend Calendar	Settlement date calculations need to know working days per country/bank	High
Currency Conversion Service	If transactions in multiple currencies, need exchange rate at transaction time	High
Archive Service	Old batches must be compressed and moved to cold storage after 90 days	High
DR/BCP Site	Disaster Recovery — system must be recoverable if primary data center fails	Critical
Health Check & Monitoring	Prometheus metrics, Grafana dashboards, PagerDuty alerts for system health	High
API Rate Limiting	Prevent abuse, protect matching engine from overload	High
Data Masking	Mask account numbers and sensitive data in non-production environments	Medium
Multi-Currency GL	Handle companies operating in multiple currencies with revaluation	Medium
Vendor Master Integration	Link transactions to vendor master to auto-identify counterparties	Medium
SLA Monitoring	Alert if batches remain unreconciled > configurable threshold (e.g., 48 hours)	Medium

Top Scalability Concerns

- Matching Engine Performance: Matching 1M transactions requires $O(n^2)$ comparisons naively. Use inverted index on amount+date to reduce to $O(n \log n)$. Pre-sort and bin transactions by date window.
- Database Write Load: Audit logs generate huge write volume. Use separate write-optimized MongoDB instance. Consider Apache Kafka for log streaming.
- File Processing: Large CSVs (>10MB) must be streamed, not loaded into memory. Use Node.js streams with backpressure.
- Report Generation: PDF generation is CPU-intensive. Run in separate worker processes. Cache generated reports — do not regenerate if data has not changed.
- Concurrent Users: During month-end close, all finance team members use system simultaneously. Connection pooling, caching, and read replicas are essential.

Production Compliance Checklist

Requirement	Standard	Implementation

Encrypt data in transit	PCI-DSS, RBI	TLS 1.3 minimum on all connections
Encrypt sensitive data at rest	PCI-DSS	AES-256 for amount, account fields; AWS KMS for key management
Access control logs	SOX, ISO 27001	Every login, every permission change, every data access logged
Data retention policy	RBI, IT Act	7-year retention with tiered storage
Business continuity	RBI PSP guidelines	RTO <4 hours, RPO <1 hour; tested quarterly
Penetration testing	PCI-DSS, RBI	Annual third-party pen test; quarterly vulnerability scans
Maker-Checker enforcement	RBI Internal Controls	Enforced at system level — cannot bypass even with admin access
Immutable audit trail	All regulations	INSERT-only DB user, hash chain, S3 Object Lock backup

Congratulations — You Are Now Ready to Build PayRecon!

This guide has taken you from zero knowledge to enterprise-grade system design. You now understand the full domain: accounting fundamentals, banking operations, reconciliation workflows, matching algorithms, audit compliance, database design, API architecture, security, AI features, and everything in between.